EMBEDDED SYSTEMS & IoT
TASK–2

Embedded Systems Lab Setup & Sensor Interfacing

Project: PIR-Based Motion Detection with Arduino UNO

Selected environment: Wokwi online simulator (or compatible Arduino UNO hardware)

1. Objective

This task develops practical understanding of an embedded development environment and sensor interfacing. The project reads a PIR motion sensor, reports its state through the Serial Monitor, and turns an LED on when motion is detected.

2. Environment and Components

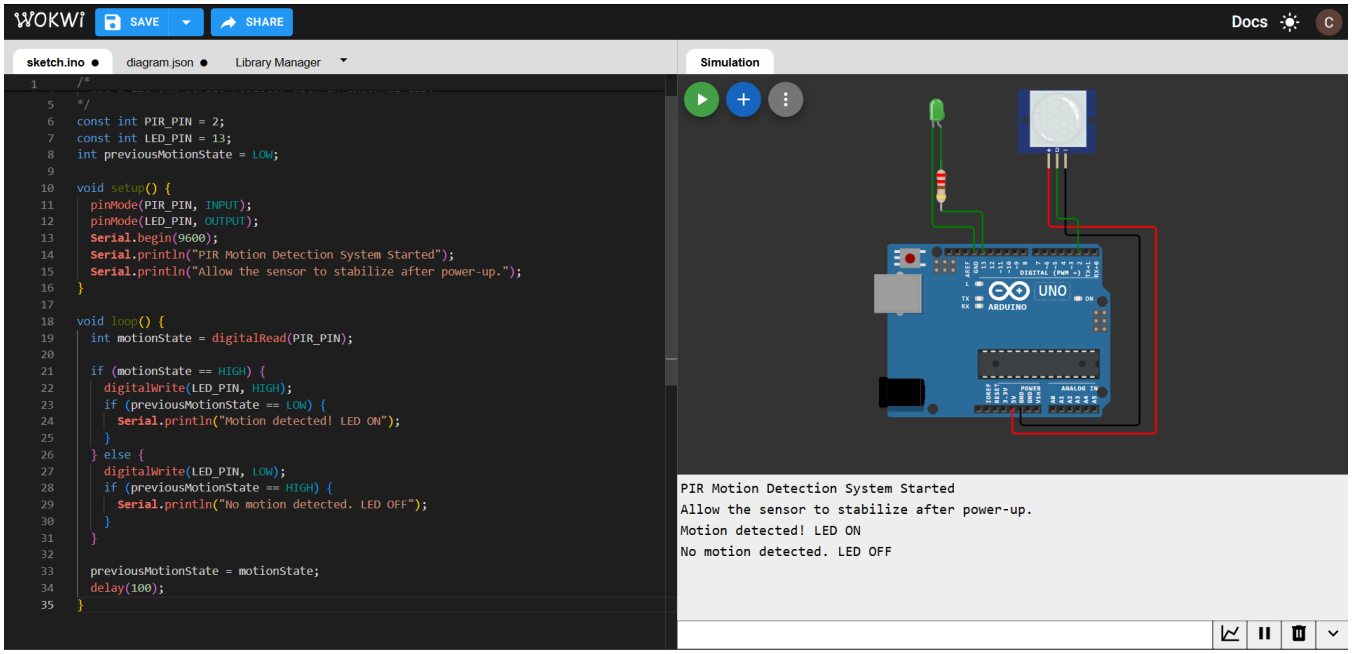
Wokwi is selected because it is a browser-based simulator for Arduino and ESP32 projects and can be used without physical hardware. The same logic can be adapted to an Arduino UNO board.

- Arduino UNO
- PIR (Passive Infrared) motion sensor
- LED and 220 Ω current-limiting resistor
- Jumper wires and breadboard for physical hardware
- Computer and Serial Monitor

3. How the PIR Sensor Works

A PIR sensor detects changes in infrared radiation in its field of view. Movement by a person or animal can change the infrared pattern received by the sensor. The module provides a digital output. In this project, HIGH is treated as motion detected and LOW as no motion currently detected. The module may need a short stabilization period after power-up.

4. Circuit Diagram and Wiring



Component pin	Arduino connection	Purpose
PIR VCC	5V	Sensor power
PIR OUT	D2	Digital motion input
PIR GND	GND	Common ground
LED anode (+)	D13 through 220 Ω resistor	Visual alert
LED cathode (−)	GND	Return path

Note: Check the labels on your exact PIR module. The Arduino UNO built-in LED on D13 can be used instead of an external LED; if so, omit the external LED and resistor.

5. Arduino Source Code

```
/*
  PIR-Based Motion Detection
  Arduino UNO: PIR OUT -> D2; LED -> D13
  Use a 220-ohm series resistor with an external LED.
*/
const int PIR_PIN = 2;
const int LED_PIN = 13;
int previousMotionState = LOW;

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);
  Serial.begin(9600);
  Serial.println("PIR Motion Detection System Started");
  Serial.println("Allow the sensor to stabilize after power-up.");
}

void loop() {
  int motionState = digitalRead(PIR_PIN);

  if (motionState == HIGH) {
    digitalWrite(LED_PIN, HIGH);
    if (previousMotionState == LOW) {
      Serial.println("Motion detected! LED ON");
```

```
    }
  } else {
    digitalWrite(LED_PIN, LOW);
    if (previousMotionState == HIGH) {
      Serial.println("No motion detected. LED OFF");
    }
  }

  previousMotionState = motionState;
  delay(100);
}
```

6. Program Explanation

- **Pin definitions:** D2 reads the PIR output and D13 controls the LED.
- **setup():** Configures the pins and starts serial communication at 9600 baud.
- **loop():** Continuously reads the digital sensor output.
- **Motion detected:** When the input is HIGH, the LED turns ON and a message is printed when the state changes.
- **No motion:** When the input is LOW, the LED turns OFF and a message is printed when the state changes back.
- **Delay:** The 100 ms delay spaces out sensor checks.

7. Testing Procedure

1. Open https://wokwi.com/ and create an Arduino UNO project.
2. Add a PIR sensor and LED; connect them according to Figure 1 and the wiring table.
3. Paste the code into the Arduino sketch and start the simulation.
4. Open the Serial Monitor and set it to 9600 baud if required.
5. Trigger motion using the simulator controls; verify the LED turns ON and a motion message appears.
6. Return the sensor to the no-motion state; verify the LED turns OFF.
7. Capture a screenshot of the working simulation. Optionally record a 20–30 second demo video.

8. Expected Output

When motion is detected, the LED turns ON and the Serial Monitor prints “Motion detected! LED ON”. When motion is no longer detected, the LED turns OFF and the monitor prints “No motion detected. LED OFF”. Timing depends on the sensor module or simulator model.

9. Applications

- Basic home-security alerts
- Motion-activated lighting
- Entry or occupancy detection
- Educational sensor-interfacing and automation projects